WMAMS System Review

Work Management & Budget Monitoring System

MoICS pilot - Architecture, Issues & Improvement Plan

Source: code analysis of wmams repo (Next.js 16 + MongoDB 7).
No PDF was found in the repo, so this document was generated from the codebase.
Date: 2026-10-06

1. What this system is

WMAMS tracks organisational units, works (projects), tasks, E-Tippani approvals, budget lines / revisions, expenditures, procurement / bids / contracts, issues / directives, documents, reports and alerts. Bilingual (EN/NE), fiscal year 2083/84, timezone Asia/Kathmandu. All business data lives in MongoDB; there is no static dataset. Pages fetch via GET /api/data/[col] (src/lib/use-data.js); writes go through PATCH/POST with __v optimistic-concurrency checks (409 on stale). First run redirects to /login; login checks the users collection (scrypt passwordHash).

Tech stack

- Stack: Framework: next 16.3.7, react 19.2.8, react-dom 19.2.8.

- DB: DB: mongoose 8.9.5, MongoDB 7 replica-set rs0.

- UI: Styling/lint: tailwindcss 4.3.3, eslint 9.39.5.

- Infra: Infra (dev): mongo:7, redis:7-alpine, minio, clamav, mailpit (infra/docker/docker-compose.dev.yml). Redis/MinIO/ClamAV are in compose but barely wired in code.

- Scripts: Scripts: dev/build/start/lint, seed via scripts/seed.mjs. Test dep mongodb-memory-server exists but no test script.

2. Architecture

```
Browser (React Client: store.js AppProvider, use-data.js, mutate.js, session.js sessionStorage)
  | fetch /api/data/:col?q&scope&page&sort + x-sid   |  fetch /api/{auth,works,tasks,engine,worker,documents}
  v
proxy.js (wmams-auth==1 cookie gate; /api/* -> 401, pages -> /login)
  v
API Routes (Next App Router, server-only: db.js -> Mongoose -> Mongo)
  |- Generic: api/data/[col]/route.js (WRITE_RULES + effectivePermsFor + canSeeAccount)
  |- Transitions: tippani/[ref]/transition, tasks/[id]/progress, budget/revisions/[id]/transition
  |- Auth: auth-mock.js (scrypt + OTP + sessions in Mongo)
  |- Jobs: engine.js (recomputeWork/rollupUnit/nightlyRun) + worker.js (9 JOBS)
  `- Docs: documents/upload-url (fake presigned URL) + clamav.js (unused there)
Pure shared libs: lifecycle.js, progress.js, metrics.js, alerts.js, rules.js, business-rules.js, scope.js
```

Key files

- DB: src/lib/db.js: cached globalThis.__wmams_mongoose, bufferCommands:false, ensureIndexes() on connect.

- Models: src/lib/models.js: COMMON {orgUnitId, orgAncestors, ...}, optimisticConcurrency:true, strict:false, 27 models.

- Scope: src/lib/scope.js: ORG_TREE, ancestorsOf, scopeFilter, PERMS_BY_ROLE, authorize() (out-of-scope = 404 = BR-03). SUPER_ADMIN has least privilege.

- Client: src/lib/use-data.js: useCollection(col) with 15s CACHE + INFLIGHT dedup, x-sid header.

3. Key flows

3.1 Auth (Sec 28)

login/page.js -> POST /api/auth/login -> auth-mock.login() scrypt verify (password.js: scrypt$salt$hash). PRIVILEGED roles get 6-digit OTP (otpchallenges TTL, console.log + devCode returned) -> verifyOtp/resendOtp -> mintSession() (single session per user, deleteMany{username}, dev.<b64>.dev access + 256-bit refresh). session.js stores sid/profile in sessionStorage + wmams-auth=1 cookie. proxy.js only checks cookie value. api-auth.js callerIdentity()/requireSuperAdmin() resolves role via getSessionIdentity(sid) (x-sid header or body.sid). Idle 30m / absolute 7d, refresh rotation with reuse detection.

3.2 Data read/write

Read: useCollection -> GET /api/data/[col]?scope=&q=&page=&sort= -> dbConnect() -> archivedAt:null filter (unless ?includeArchived=1 + org.unit.manage) -> orgAncestors match -> buildSearchFilter() regex -> SMALL_IN_MEMORY {users, roles, lookups...} in-memory vs skip/limit+count -> stripSecrets. Write: mutate.js (x-sid+body.sid) -> POST{doc}/PATCH{match,update,__v} -> denyWrite(WRITE_RULES) + per-col gates -> checkVersion(__v)->409 -> Object.assign+save().

3.3 Lifecycle / progress / engine

lifecycle.js: WORK_TRANSITIONS + LIFECYCLES {task, progressUpdate, tippani, expenditure, budgetRevision, procurement, contract, issue, directive}, canTransition/applyTransition (history+audit). Only tippani/transition and tasks/progress enforce it server-side; generic data/[col] PATCH mostly bypasses. progress.js: calcExpected/calcPhysical {WEIGHTED_TASKS, QUANTITY, MILESTONE, HYBRID, LEAF} -> engine.js: enqueueRecompute / recomputeWork / rollupUnit / nightlyRun -> metrics.js: enrichWorks/delayedWorks/avgPhysical/utilisation -> alerts.js: 16 ALERT_RULES + digests.

4. Collections (models.js: COLLECTIONS)

units, users, roles (RolePerms), works, tasks, tippanis, budget (BudgetLine), expenditures, procurements, contracts, bids, issues, directives, docs, members, budgetrevisions, auditlogs, lookups, documents, snapshots, sessions, otpchallenges (TTL), importbatches, trimester, reports, reportschedules. All with orgAncestors multikey index (SCOPE_IDX).

5. Issues found (grouped by severity)

P0 - Security (fix first)

- 1. Forgeable gate: src/proxy.js:16-18, src/lib/session.js:13-14, src/lib/auth-mock.js:210. Any cookie value passes all /api/* (except public prefixes). Non-HttpOnly cookies + unsigned dev.xxx.dev access tokens.

- 2. MFA secret leaks: src/lib/auth-mock.js:129,148,198, src/app/api/auth/login/route.js:9. login/resend/step-up return it and log to console. OTP bypassable in dev.

- 3. Reads ignore session scope (BR-03 bypass): src/app/api/data/[col]/route.js:180-186. No callerIdentity on large-col GET. Any authed user enumerates any unit by guessing scope.

- 4. Dead req.method check disables POST gates: route.js:309,320. (req.method==POST always false in App Router) so validateRoleDoc/LOCKED_ROLES and units SUPER_ADMIN checks never run on POST.

- 5. Mass-assignment + lifecycle bypass: models.js:17 strict:false + route.js:531 Object.assign. Client can overwrite orgAncestors/status/role/committed; all WORK_TRANSITIONS skippable except works->VERIFIED. tippani PATCH vs transition route diverge.

P1 - Data / money correctness

- 6. Non-atomic money commit: src/app/api/tippani/[ref]/transition/route.js:94-101. Two save()s (budget.committed+= then tip.committed=true) with no transaction/unique guard; concurrent approves double-commit.

- 7. Broken concurrency/idempotency: src/lib/api-helpers.js:86. Missing __v defaults to 1 (always passes); idempotency() never

called though client sends idempotencyKey.

- 8. Unvalidated schemas: models.js. No required fields; ensureIndexes warns-and-continues, so duplicate id/ref/voucher slip on cold start.

- 9. Audit gaps + enumeration: auth-mock.js:277, business-rules.js:41. Audit only in tippani/task routes, generic PATCH writes none; AuditLog has no index/immutability; UNKNOWN_USER admits enumeration; pwHistory uses plaintext in-memory.

P2 - Fake / in-memory wiring

- 10. Fake document pipeline: upload-url/route.js:14,54. Fake https://*.example presigned URL, in-memory seq id, trusts client size/mime; clamav.js never called; no S3 wiring despite worker virus-scan job.

- 11. Ephemeral security state: auth-mock.js:38-48, engine.js:13, api-helpers.js:94 seenKeys. Lost on restart, divergent across instances despite Redis in compose.

- 12. Hardcoded secrets: env.js:27-47, docker-compose.dev.yml:28,42, integration.js:44. Dev JWT/S3/encryption fallbacks, app:devpasswordchange, minioadmin-change-me, same URI in seed; SMS log provider.

P3 - Scale, logic, DevOps

- 13. Scalability traps: data/[col]/route.js:194, api-helpers.js:67, worker/route.js:12, use-data.js:19. SMALL_IN_MEMORY loads whole users/roles; JSON.stringify(row).includes(q) full-scan (q dropped for small cols); worker snapshot() loads 7 collections; 15s stale cache.

- 14. Logic/calendar bugs: progress.js:92, business-rules.js:20, rules.js:159, metrics.js:14. calcPhysicalHybrid missing parens (only wM term /100); isBsDate allows day 32; bsToAd 3 hardcoded dates; toISOString UTC vs Kathmandu.

- 15. Client-trusted identity: store.js:57-61, tasks/[id]/progress/route.js:43. UI role-switcher setRole() + permsFor() for gating; tasks/progress trusts body.scopeUnitIds; upload-url uses static permsFor not DB effectivePermsFor.

- 16. DevOps void: health/route.js:18, api-manifest.js:25-36. No Dockerfile/prod compose/CI/tests/migrations; health exposes works count; duplicate live/planned routes.

6. Where to improve (roadmap)

Phase 0 - Security (1-2 weeks, highest ROI)

- Auth + scope: HttpOnly Secure signed session cookie; verify session on every API route; derive scope server-side, never trust ?scope=. Remove devCode from responses/logs.

- Writes: Fix POST gate (use request.method), whitelist updatable fields per collection, enforce lifecycle.canTransition in generic PATCH, strip orgAncestors/status/role from client input.

- State + secrets: Move stepTokens/resetTokens/seenKeys/queue to Mongo or Redis; rotate dev secrets via env.

Phase 1 - Correctness (2-3 weeks)

- Money + concurrency: Wrap budget commit in Mongo transaction with idempotency key unique index; require __v on PATCH; wire idempotency() helper.

- Schemas: Set strict:true or strict:throw, add required + unique indexes that fail hard on boot; validate with zod/valibot on POST/PATCH.

- Audit + passwords: Write audit log on every mutation (actor, col, id, diff); make AuditLog append-only; fix UNKNOWN_USER to generic invalid credentials; add password complexity + history hash.

Phase 2 - Real integrations

- Docs: Presigned MinIO/S3 URLs server-side, enforce size/mime server-side, call clamav.scanUpload before complete; store scan verdict.

- Infra: Wire Redis for queue/cache, Ncell/NTC or email provider via integration.js, BS calendar library instead of hardcoded

dates.

Phase 3 - Scale + UX

- Perf: Remove SMALL_IN_MEMORY full loads; add text indexes + server pagination/sort; use SWR invalidation on mutate instead of fixed 15s; paginate worker snapshot job.

- Bugs: Fix hybrid parens, BS validation, Kathmandu timezone (use Asia/Kathmandu, not toISOString); server-compute effectivePermsFor for UI.

- DevOps: Add Dockerfile + prod compose, GitHub Actions (lint/build/test), mongodb-memory-server tests for scope/transitions/money, migration script, backup/restore docs.

7. Suggested file-by-file actions

- src/proxy.js, src/app/api/data/[col]/route.js: Verify cookie session, call callerIdentity(), derive scopeFilter from session; fix POST branch.

- src/lib/models.js, src/lib/db.js: strict:true, required:true, unique indexes, throw on ensureIndexes failure.

- src/lib/auth-mock.js: Remove devCode, persist tokens in Mongo with TTL, hash pw history.

- src/app/api/tippani/[ref]/transition/route.js, src/app/api/engine/route.js: Use transactions + unique idempotency key.

- src/app/api/documents/upload-url/route.js, src/lib/clamav.js: Real S3 presign + scanUpload gate.

- src/lib/progress.js, metrics.js, business-rules.js: Fix formula parens, timezone, BS validation.

- repo root / infra/: Add tests, CI, Dockerfile, prod compose.

8. Bottom line

This is a well-scoped pilot with clean separation (proxy gate, generic data API, pure scope/lifecycle/metrics libs). It is not yet production-safe: authz is client-trustable, writes bypass lifecycle, money commits are non-atomic, and docs/infra are stubbed. Do Phase 0 + Phase 1 before any real budget data. Estimated order: gate/scope fix (days) > write whitelist + transactions (1-2 weeks) > schemas/audit/tests (2 weeks) > integrations/scale (ongoing).